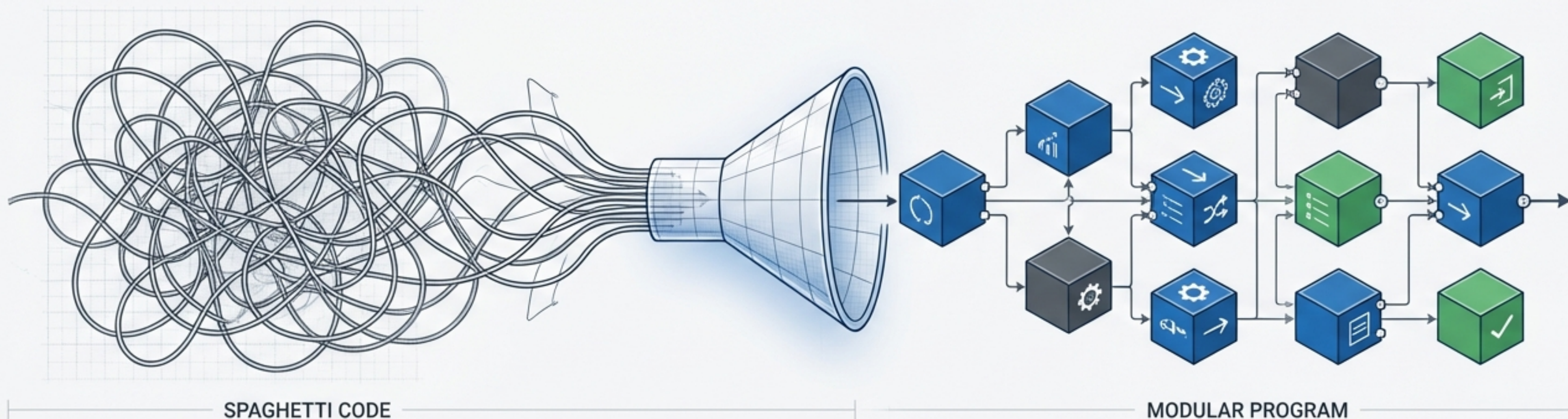


C语言函数：构建程序的基石



从混乱到模块化，探索C语言中最重要的构件块。

函数是什么？为何需要函数？

函数是什么？为何需要函数封装，复用、抽象中最重要的构件块。



封装



复用



抽象

函数如何通信：参数与返回值



⚠ 可能通信通信，可能可活性问题

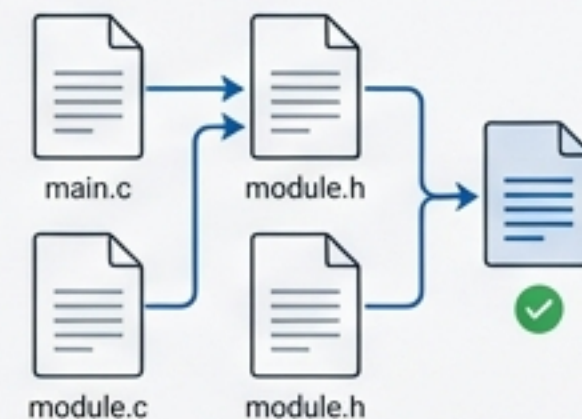
安全保障：函数原型



高级技巧：递归与指针



组织大型程序：多文件编译



为何需要函数？直面代码的复杂度

"main()"

```
int main(void) {  
    // 读入一系列数字...  
    // (大量代码)  
    // 分类这些数字...  
    // (大量重复代码)  
    // 找出这些数字的平均值...  
    // (大量代码)  
    // 打印一份柱状图...  
    // (大量代码)  
    // 再次读入另一系列数字...  
    // (重复的读入代码)  
    // ...  
}
```



难以阅读：逻辑混杂，难以理解程序流程。



代码重复：同样的功能需要在多处复制粘贴。



难以调试：一个小错误可能影响整个程序，难以定位。



难以维护：修改一个功能风险极高。

“黑盒”化思维：模块化与复用

“许多程序员喜欢把函数看作是...‘黑盒’。只需知道给该函数传入信息（输入）及其生成的响应（输出），无需了解其内部代码。”

"main()"

```
int main(void) {  
    float list[SIZE];  
  
    readlist(list, SIZE);  
    sort(list, SIZE);  
    average(list, SIZE);  
    bargraph(list, SIZE);  
  
    return 0;  
}
```

readlist()



sort()



average()



bargraph()



模块化 (Modularity)：每个函数完成一个特定任务。

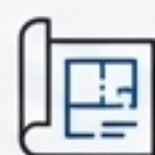


可复用 (Reusability)：`sort()`函数可以在其他程序中再次使用。



专注整体设计 (Focus on Design)：程序员可以专注于程序整体结构，而非实现细节。

函数的三个面孔：原型、调用与定义



原型 (The Blueprint)

```
void starbar(void);
```

“函数原型 (function prototype) 告诉编译器函数的类型、参数和返回值。这是函数的‘说明书’。”



调用 (The Usage)

```
// in main()  
starbar();
```

“函数调用 (function call) 表明在此处执行函数。”



定义 (The Mechanics)

```
void starbar(void) {  
    int count;  
    for (count = 1; ...)   
        putchar('*');  
    putchar('\n');  
}
```

“函数定义 (function definition) 明确地指定了函数要做什么。这是‘黑盒’的内部实现。”

传递信息：参数 (Passing Information In)

Key Terminology

形式参数 (Formal Parameters)：函数定义时声明的变量（`char ch`，`int num`），它们是函数内部的局部变量。

实际参数 (Actual Arguments)：函数调用时传递的真实值（`'\*'`，`40`）。

主调函数 (Caller)

```
main()
```

```
show_n_char('*', WIDTH);
```

(value: 40)
实际参数 (Actual Arguments) 40

被调函数 (Called Function)

```
void show_n_char(char ch, int num)
```

ch

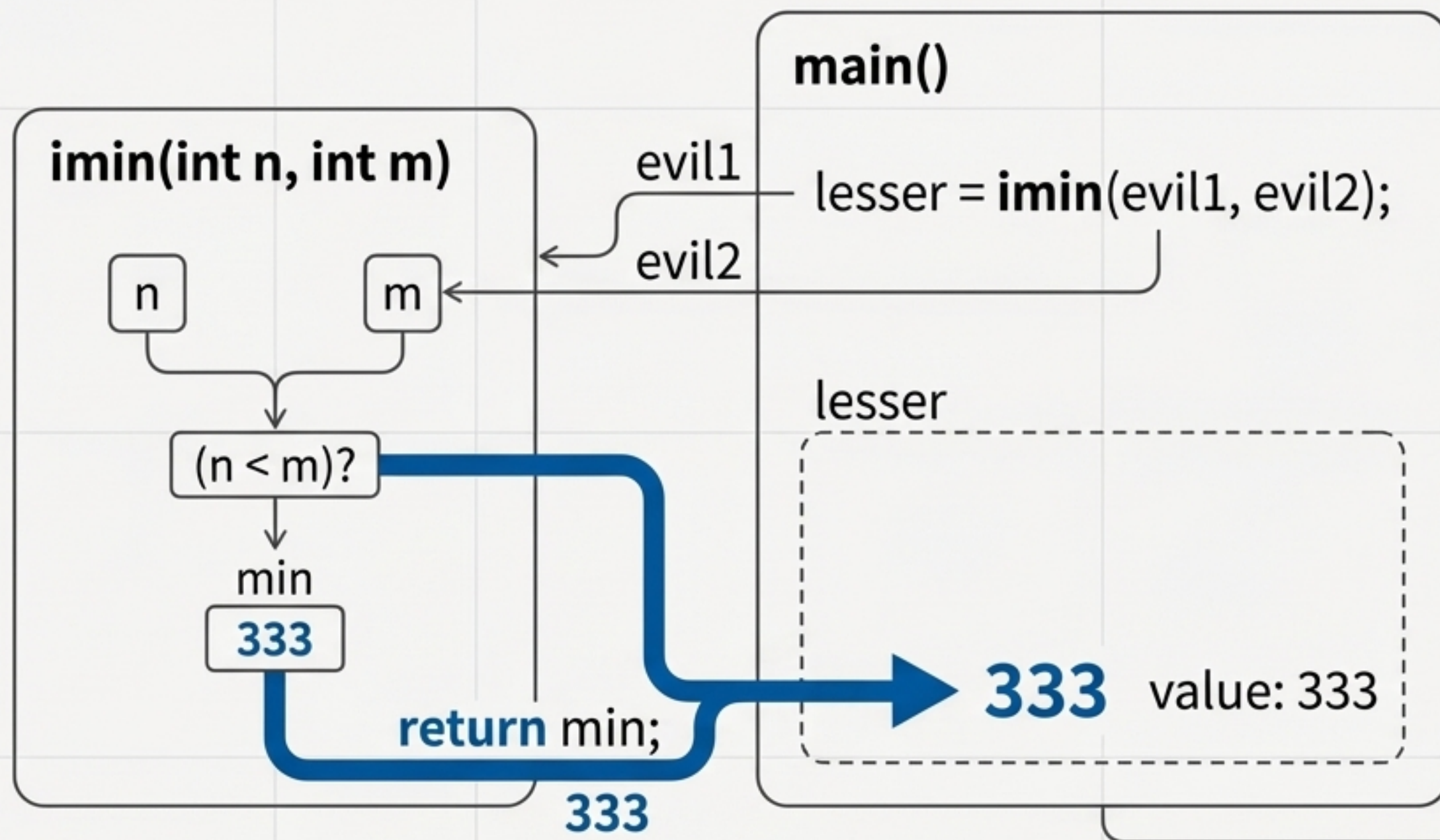
形式参数
(Formal Parameters)

num

形式参数
(Formal Parameters)

“调用函数时，实际参数的值被**拷贝**给被调函数相应的形式参数。被调函数对拷贝数据的操作，不会影响主调函数中的原始数据。”

返回结果：`return` 关键字 (Getting Results Back)



简洁版本

```
int imin(int n, int m) {  
    return (n < m) ? n : m;  
}
```

返回值可以是任意表达式。

“**return** 语句不仅返回值，也立即终止函数的执行，将控制权交还给主调函数。

没有“图纸”的风险：旧式声明的严重问题

```
// Old-style Declaration (Danger!)  
int imax(); /* 未指明参数数量和类型 */
```

```
// Erroneous Calls (Undefined Behavior)  
imax(3)  
imax(3.0, 5.0)
```

错误输出显示 (INCORRECT OUTPUT)

The maximum of 3 and 5 is 1606416656. ✖

The maximum of 3 and 5 is 3886. ✖



“编译器在‘盲飞’！主调函数按自己的方式压栈，被调函数按另一种方式读栈，导致数据错乱。”

ANSI C原型：编译器的“连接”检查器

没有原型

```
// Old-style Declaration  
int imax(); /* 未指明参数数量和类型 */
```

```
// Erroneous Calls  
imax(3.0, 5.0);
```

DANGER

• The maximum of 3 and 5 is 1606416656. ❌

使用ANSI C原型

```
// Correct Prototype  
int imax(int, int);
```

```
imax(3)
```

⚠️ 错误：参数数量太少 (too few arguments)

```
imax(3.0, 5.0)
```

自动类型转换

```
imax(3, 5)
```

The maximum of 3 and 5 is 5. ✅

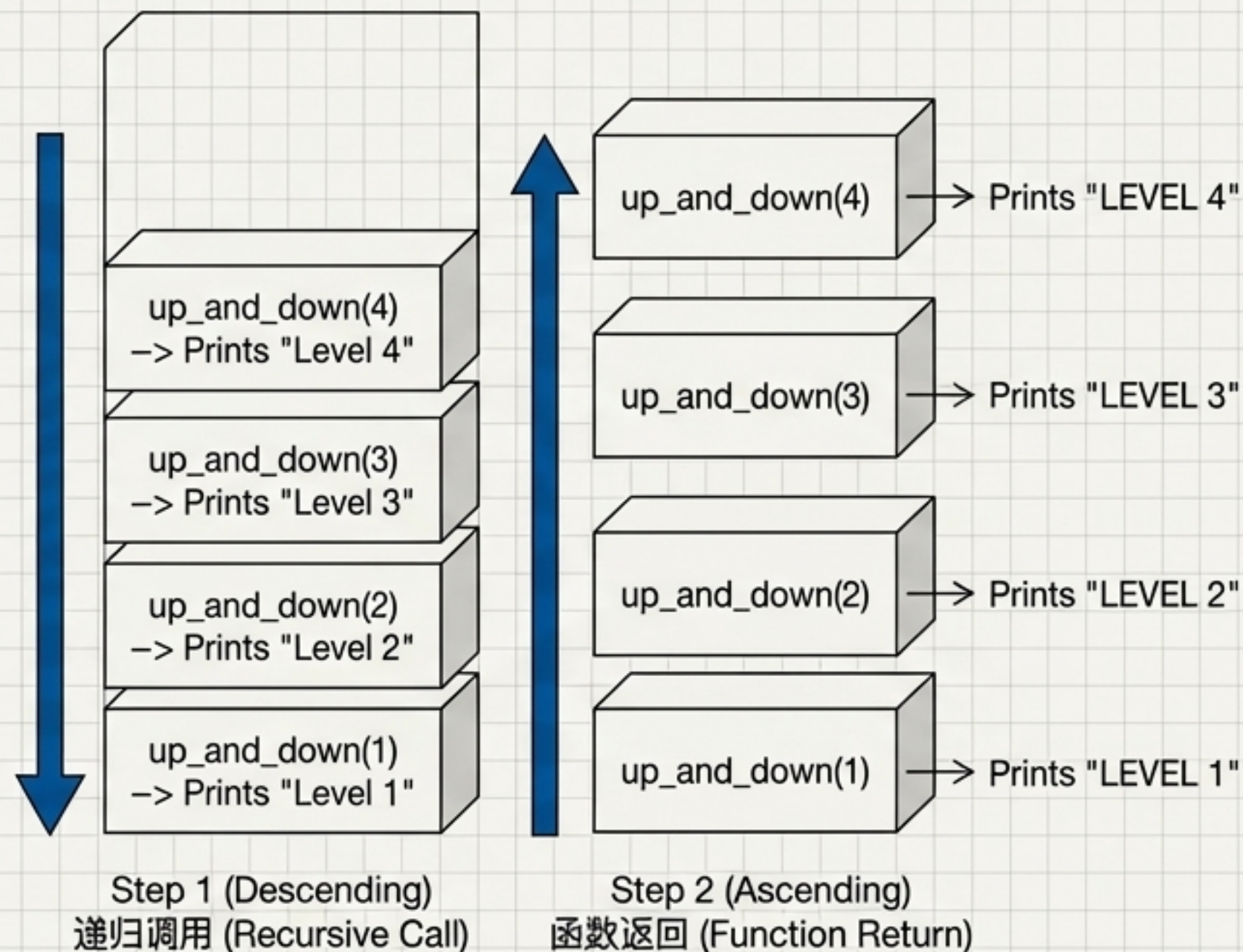
函数原型是C语言的强力工具，它让编译器能捕获许多在运行时难以发现的错误。

当函数调用自身：递归的逻辑

```
void up_and_down(int n) {  
    printf("Level %d\n", n); // 递归调用前  
    if (n < 4)  
        up_and_down(n + 1);  
    printf("LEVEL %d\n", n); // 递归调用后  
}
```

Output

Level 1
Level 2
Level 3
Level 4
LEVEL 4
LEVEL 3
LEVEL 2
LEVEL 1

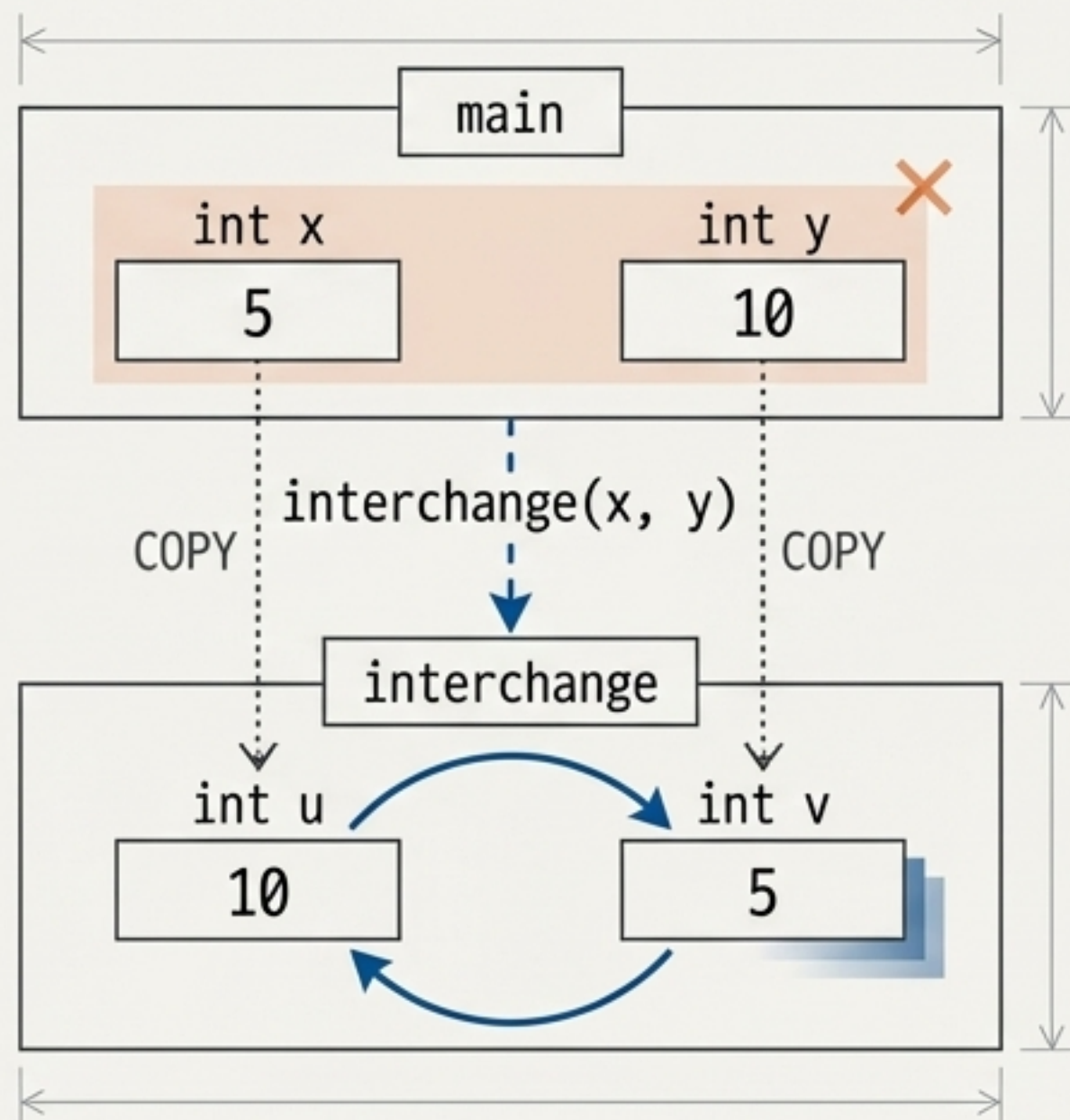


① 递归调用之前的语句按顺序执行，之后的语句按相反顺序执行。

挑战：函数如何才能修改“外部世界”？

```
// swap1.c
void interchange(int u, int v) {
    int temp = u;
    u = v;
    v = temp;
}

int main() {
    int x = 5, y = 10;
    printf("Originally x = %d and y = %d.\n", x, y);
    interchange(x, y);
    printf("Now x = %d and y = %d.\n", x, y);
    return 0;
}
```



Output:
Originally x = 5 and y = 10.
Now x = 5 and y = 10. 

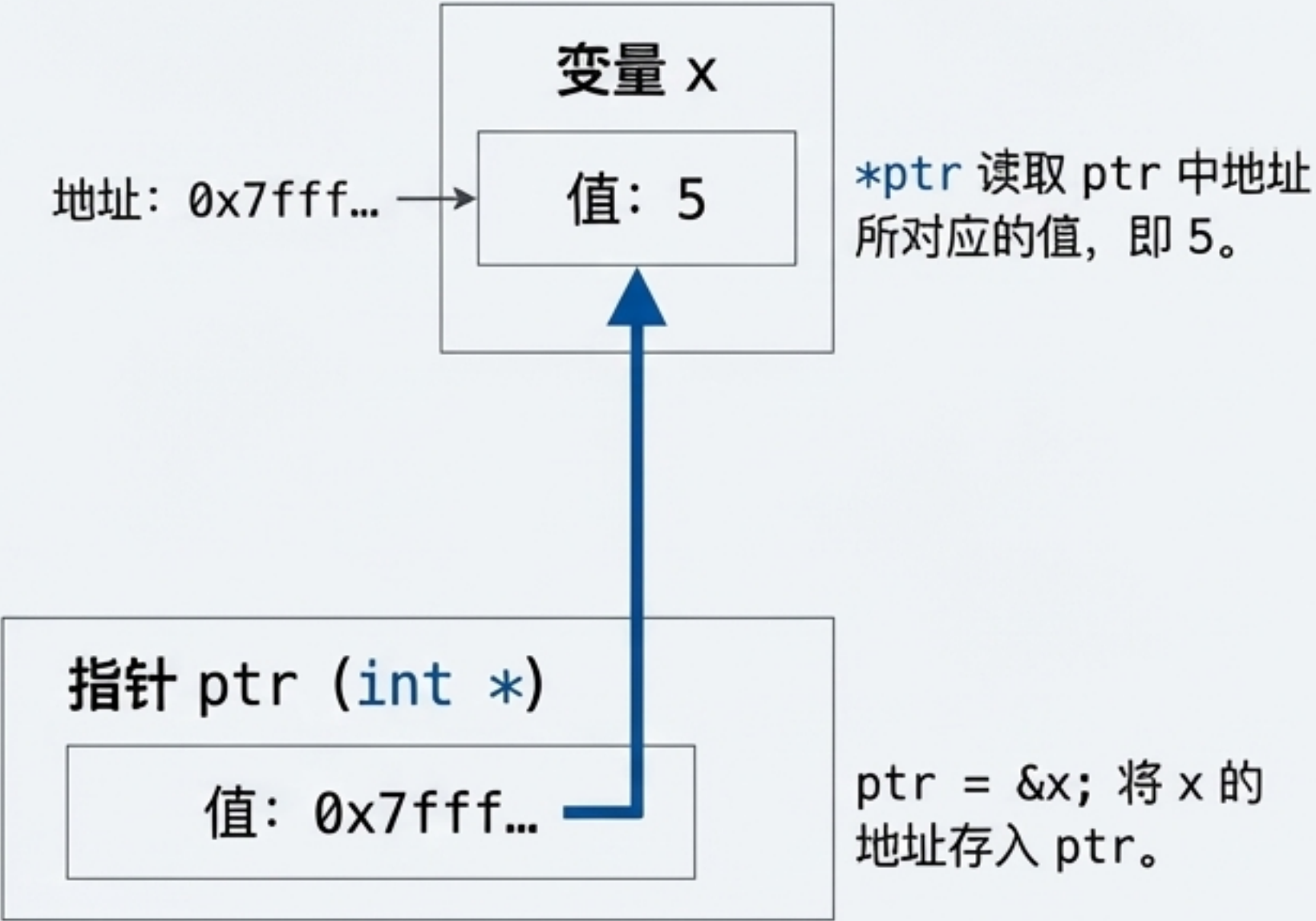
为何交换失败了？因为 `interchange` 函数操作的只是 `x` 和 `y` 的副本。
我们需要一种方法，让函数能直接操作 `main` 函数的变量。



解决方案：传递地址，而非值

指针 (Pointer) 是一个值为内存地址的变量。

关键运算符	
&	地址运算符 获取变量的内存地址。 &pooh → 变量 pooh 的地址。
	间接/解引用运算符 获取指针所指向地址上储存的值。val = *ptr; → val 获取 ptr 所指向地址上的值。



```
int * pi; // pi 是一个指针, 它指向一个 int 类型的变量
// 关键解读: int * pi; 的意思是 *pi 的类型是 int.
```


指针的力量：在函数内修改主调函数变量

Corrected Code

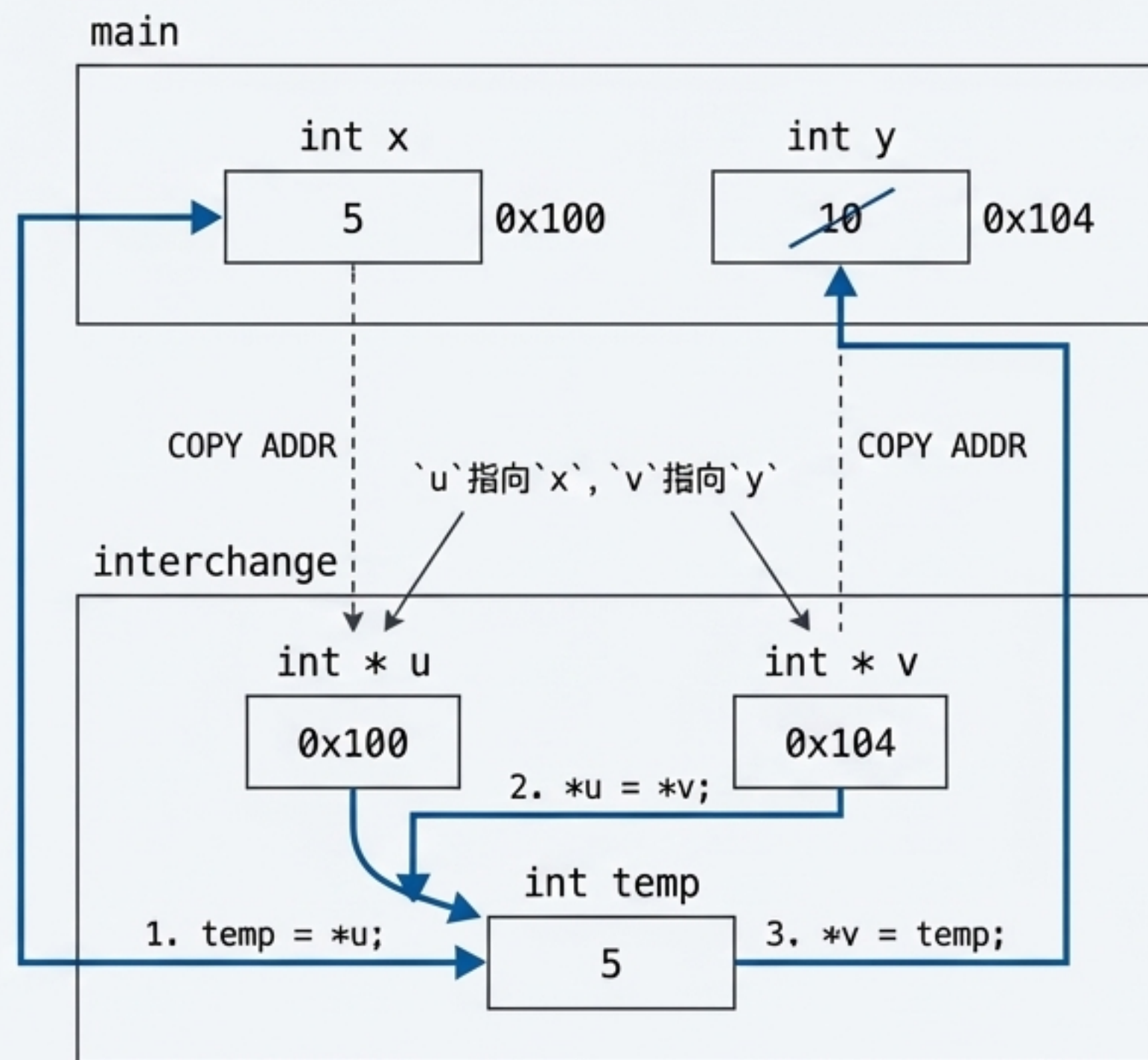
```
// 调用时传递地址
interchange(&x, &y);

// 函数定义中使用指针作为参数
void interchange(int * u, int * v) {
    int temp;
    temp = *u; // temp 获得 u 指向对象(x)的值
    *u = *v; // u 指向对象(x)被赋予 v 指向对象(y)的值
    *v = temp; // v 指向对象(y)被赋予 temp 的值
}
```

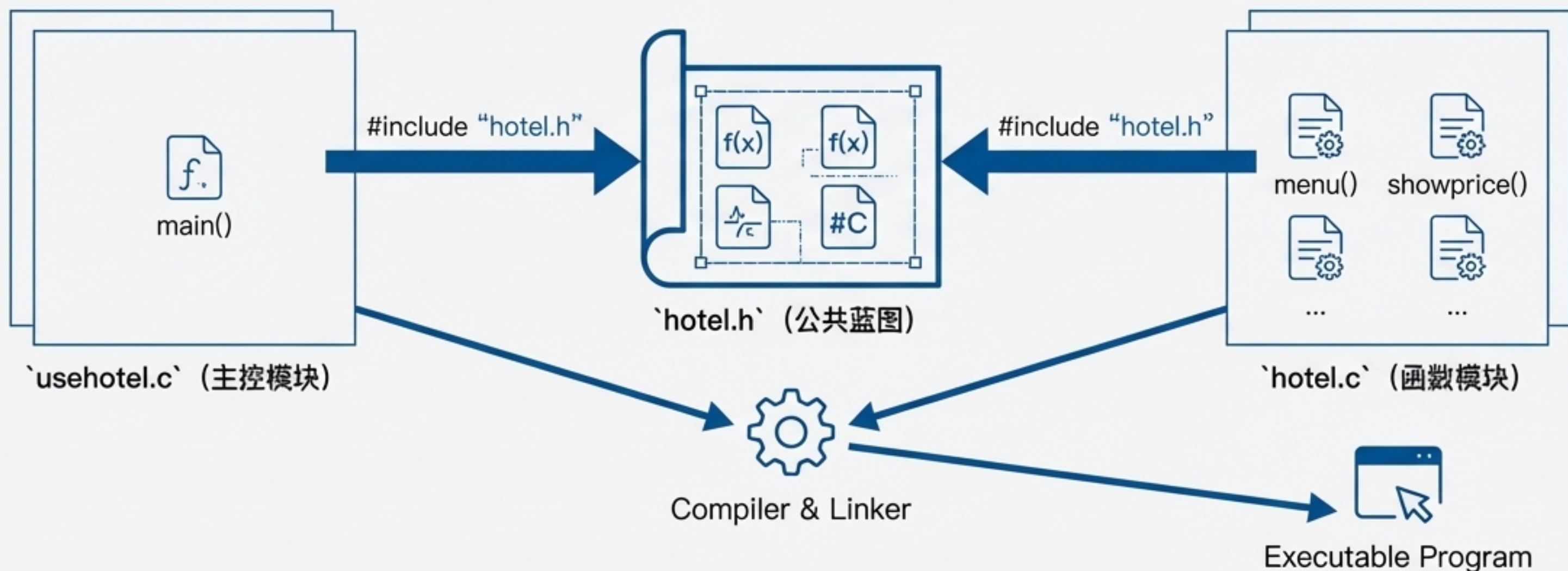
Successful Output

Originally x = 5 and y = 10.

Now x = 10 and y = 5. ✓



组装程序：多文件编译与头文件



`hotel.h`

- 包含函数原型: `int menu(void);` 等。
- 包含符号常量: `#define HOTEL1 180.00` 等。

`hotel.c`

- 提供 `menu()`, `getnights()`, `showprice()` 的具体实现。
- 包含 `#include "hotel.h"`。

`usehotel.c`

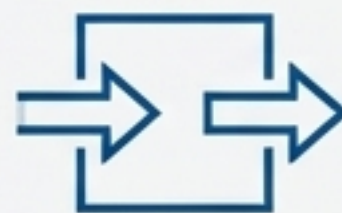
- 包含 `main()` 函数, 负责调用其他模块的函数。
- 包含 `#include "hotel.h"`。

核心概念回顾



模块化 (Modularity)

将函数视为独立的“黑盒”，每个黑盒完成一项特定任务。



接口 (Interface)

通过参数向函数传递信息（输入），通过 `return` 语句从函数获取结果（输出）。



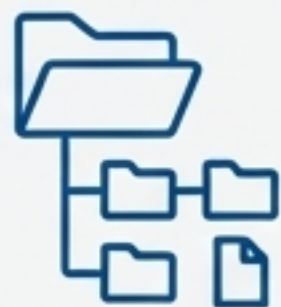
契约 (Contract)

函数原型是编译器和程序员之间的契约，确保类型安全和正确连接。



高级技巧 (Advanced Techniques)

递归用于解决自相似问题；指针用于在函数间共享和修改数据。



组织 (Organization)

使用头文件共享原型和常量，将大型程序组织成逻辑分离、易于维护的多个文件。

“所有C函数，生而平等。”

程序中的每个C函数与其他函数都是平等的。每个函数都可以调用其他函数，或被其他函数调用。`main()`函数的特殊之处仅在于它是程序的入口点。

掌握函数，就是掌握了构建可靠、优雅且强大软件的艺术。